Estudio comparativo de algoritmos de optimización para el entrenamiento de redes neuronales convolucionales en la clasificación de tumores cerebrales en MRI

Proyecto final – MEMAD · Entrega I

Alumno: [Nombre del alumno] · Código: [Código]
Profesor: Dr. E. Alejandro Guerrero Arroyo · CUCEI, Universidad de Guadalajara
Fecha de entrega: 9 de noviembre de 2026

1. Planteamiento

El proyecto usa la clasificación de resonancias magnéticas cerebrales en cuatro clases (glioma, meningioma, tumor de hipófisis y sin tumor) como **caso de estudio** para analizar algoritmos de optimización. El foco no es obtener la mayor exactitud posible, sino comparar cómo distintos algoritmos minimizan la función de pérdida: velocidad de convergencia, estabilidad, sensibilidad a la tasa de aprendizaje y capacidad de generalización.

Objetivos

- **O1.** Implementar desde cero optimizadores de primer orden (SGD, momento, Nesterov, Adagrad, RMSprop, Adam, AdamW) y validarlos numéricamente contra PyTorch.
- **O2.** Comparar estos optimizadores entrenando una CNN **desde cero** (sin pesos preentrenados), con un protocolo justo: mismo presupuesto de épocas, búsqueda de tasa de aprendizaje por algoritmo y varias semillas.
- **O3.** Estudiar un problema **convexo** (regresión logística multiclase sobre características extraídas de la red) y comparar métodos de primer y segundo orden (descenso de gradiente, gradiente conjugado, Newton, L-BFGS), contrastando las tasas de convergencia teóricas con las observadas.
- **O4.** Analizar la programación de la tasa de aprendizaje y la optimización de hiperparámetros como problema de caja negra (búsqueda aleatoria frente a optimización bayesiana).
- **O5.** Integrar el mejor modelo en una aplicación web de demostración.

2. Cronograma de metas y entregables

Entrega	Metas de la semana	Entregables
I 9 nov	• Planteamiento, objetivos y cronograma. • Análisis del dataset y partición train / validación / test. • Modelo de referencia (ResNet-50 con *transfer learning*). • Implementación propia de 6 optimizadores y validación contra `torch.optim`. • Primer experimento: trayectorias en funciones de prueba 2D (cuadrática mal condicionada y Rosenbrock).	• Este PDF (cronograma, ingredientes y avance). • `optimizadores.py`, `test_optimizadores.py`, `trayectorias_2d.py`. • Figuras y tablas de resultados. • `train_resnet50.py` (modelo de referencia).
II 16 nov	• Marco de experimentación común: misma red (ResNet-18 sin preentrenar), mismos datos, registro automático de métricas. • *LR range test* (Smith, 2017) para elegir la tasa de aprendizaje de cada optimizador. • Comparación de los 7 optimizadores × 3 semillas.	• Código del marco de experimentos. • Curvas de pérdida y exactitud por época. • Tabla media ± desviación estándar (validación y test), tiempo por época.
III 23 nov	• Extracción de características con la red congelada (con reducción de dimensión por PCA). • Regresión logística multiclase con regularización L2 (problema convexo). • Implementación y comparación de descenso de gradiente, gradiente conjugado no lineal, Newton y L-BFGS. • Verificación de tasas de convergencia (lineal, superlineal, cuadrática).	• Código de los métodos. • Gráficas de $\|\nabla f\|$ y $f - f^*$ por iteración y por tiempo. • Discusión teoría vs. práctica.
IV 30 nov	• Programación de la tasa de aprendizaje: constante, escalonada, coseno y OneCycle. • Optimización de hiperparámetros: búsqueda aleatoria vs. bayesiana (TPE, Optuna) con el mismo número de pruebas. • Análisis de errores clínicos (falsos negativos de glioma).	• Código y resultados de ambos estudios. • Curvas de "mejor valor encontrado vs. número de pruebas".
V 7 dic	• Entrenamiento del modelo final con la mejor configuración. • Aplicación web de demostración (Gradio). • Informe final y conclusiones.	• Informe final (PDF) y presentación. • Código completo y aplicación web.

Si el tiempo lo permite, se añadirá un análisis del paisaje de la función de pérdida y del optimizador SAM (Foret et al., 2021) en la semana IV.

3. Ingredientes del proyecto

Datos

Recurso	Enlace	Uso
Brain Tumor MRI Dataset (M. Nickparvar)	kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset	Dataset principal: 7 200 imágenes, 4 clases.
Fuente original: figshare (Cheng et al.)	figshare.com/articles/dataset/brain_tumor_dataset/1512427	Incluido en el dataset principal.
Fuente original: SARTAJ	kaggle.com/datasets/sartajbhuvaji/brain-tumor-classification-mri	Incluido en el dataset principal.
Fuente original: Br35H	kaggle.com/datasets/ahmedhamada0/brain-tumor-detection	Imágenes sin tumor del dataset principal.

Modelos

Recurso	Enlace	Uso
ResNet-50 preentrenada en ImageNet (torchvision, pesos `IMAGENET1K_V2`)	pytorch.org/vision/…/torchvision.models.resnet50.html Archivo: download.pytorch.org/models/resnet50-11ad3fa6.pth	Modelo de referencia y extractor de características (semana III).
ResNet-18 (sin pesos preentrenados)	pytorch.org/vision/…/torchvision.models.resnet18.html	Red entrenada desde cero para comparar optimizadores.

Software y archivos

Recurso	Enlace	Uso
Código del proyecto	github.com/gabriel73-art/DataProcessor (carpeta brain_tumor)	Todos los scripts y resultados.
PyTorch y torchvision	pytorch.org	Redes, diferenciación automática, optimizadores de referencia.
Google Colab (GPU)	colab.research.google.com	Entrenamiento.
SciPy	scipy.org	Referencia para L-BFGS y gradiente conjugado (semana III).
Optuna	optuna.org	Optimización bayesiana de hiperparámetros (semana IV).
Gradio	gradio.app	Aplicación web (semana V).

Referencias bibliográficas

- Kingma, D. P. & Ba, J. (2015). Adam: A Method for Stochastic Optimization. ICLR. arxiv.org/abs/1412.6980
- Loshchilov, I. & Hutter, F. (2019). Decoupled Weight Decay Regularization. ICLR. arxiv.org/abs/1711.05101
- Duchi, J., Hazan, E. & Singer, Y. (2011). Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. JMLR 12. jmlr.org/papers/v12/duchi11a.html

- Sutskever, I., Martens, J., Dahl, G. & Hinton, G. (2013). On the importance of initialization and momentum in deep learning. ICML. proceedings.mlr.press/v28/sutskever13.html
- Tieleman, T. & Hinton, G. (2012). Lecture 6.5 – RMSprop. Neural Networks for Machine Learning, Universidad de Toronto. cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf
- He, K., Zhang, X., Ren, S. & Sun, J. (2016). Deep Residual Learning for Image Recognition. CVPR. arxiv.org/abs/1512.03385
- Smith, L. N. (2017). Cyclical Learning Rates for Training Neural Networks. WACV. arxiv.org/abs/1506.01186
- Smith, L. N. & Topin, N. (2019). Super-Convergence. arxiv.org/abs/1708.07120
- Foret, P. et al. (2021). Sharpness-Aware Minimization for Efficiently Improving Generalization. ICLR. arxiv.org/abs/2010.01412
- Nocedal, J. & Wright, S. J. (2006). *Numerical Optimization*, 2.ª ed. Springer.

4. Avance de la semana I

4.1 Dataset y partición

Las clases están equilibradas. Del conjunto *Training* se separa un 15 % estratificado (semilla fija) como validación; el conjunto *Testing* solo se usa una vez, al final.

Conjunto	Por clase	Total	Uso
Entrenamiento	1 190	4 760	Ajuste de los pesos.
Validación	210	840	Selección del mejor modelo, *early stopping* e hiperparámetros.
Prueba	400	1 600	Evaluación final.

4.2 Modelo de referencia (ResNet-50 con *transfer learning*)

ResNet-50 preentrenada en ImageNet y ajustada completa, con tasas de aprendizaje discriminativas por bloque (1e-5 a 1e-3), AdamW, *label smoothing* 0.1, programación OneCycle y 20 épocas en GPU. Sirve como cota de referencia para lo que se logra con una red preentrenada.

Métrica	Valor
Mejor exactitud en validación (época 18)	99.64 %
Exactitud en prueba	**96.00 %**

Real \ Predicho	glioma	meningioma	sin tumor	hipófisis	Exactitud por clase
glioma	338	32	29	1	84.50 %
meningioma	0	398	0	2	99.50 %
sin tumor	0	0	400	0	100.00 %
hipófisis	0	0	0	400	100.00 %

Observaciones: (1) no hay sobreajuste clásico, ya que las curvas de entrenamiento y validación avanzan juntas; (2) 62 de los 64 errores son gliomas, lo que sugiere una diferencia de distribución entre los gliomas de *Training* y de *Testing*; (3) los 29 gliomas clasificados como "sin tumor" son falsos negativos, el error clínicamente más grave, y se analizarán en la semana IV.

4.3 Implementación propia de optimizadores

Se implementaron como subclases de `torch.optim.Optimizer` (archivo `optimizadores.py`), siguiendo las ecuaciones originales:

```
SGD: θ ← θ − η g
Momento: v ← μ v + g ; θ ← θ − η v
Nesterov: v ← μ v + g ; θ ← θ − η (g + μ v)
Adagrad: G ← G + g² ; θ ← θ − η g / (√G + ε)
RMSprop: s ← α s + (1−α) g² ; θ ← θ − η g / (√s + ε)
Adam: m ← β₁m + (1−β₁)g ; v ← β₂v + (1−β₂)g² ; m̂ = m/(1−β₁ᵗ) ; v̂ = v/(1−β₂ᵗ) ; θ ← θ − η m̂ / (√v̂ + ε)
AdamW: θ ← θ (1 − ηλ) antes del paso de Adam (weight decay desacoplado)
```

Validación. Para cada algoritmo se entrenó la misma CNN pequeña (misma inicialización y mismos datos) durante 200 pasos con nuestra implementación y con la de PyTorch, en doble precisión.

Optimizador	**máx $\lvert\theta_{propio} - \theta_{PyTorch}\rvert$**	**Pérdida inicial → final**	**Resultado**
SGD	0	1.5138 → 1.3860	OK
SGD + momento	0	1.5138 → 1.3867	OK
Nesterov	1.8×10^{-16}	1.5138 → 1.3860	OK
SGD + momento + L2	1.1×10^{-16}	1.5138 → 1.3866	OK
Adagrad	0	1.5138 → 1.3824	OK
RMSprop	0	1.5138 → 1.3878	OK
Adam	1.1×10^{-16}	1.5138 → 1.3907	OK
Adam + L2	5.6×10^{-17}	1.5138 → 1.3926	OK
AdamW	5.6×10^{-17}	1.5138 → 1.3908	OK

Todas las diferencias están en el orden del error de redondeo de doble precisión ($\approx 10^{-16}$). Como control, al eliminar a propósito la corrección de sesgo de Adam, la prueba falla con diferencias del orden de 10^{-2}, lo que confirma que la validación detecta errores reales. (La pérdida apenas baja porque los datos de la prueba son aleatorios; el objetivo aquí es solo verificar la equivalencia numérica.)

4.4 Experimento: funciones de prueba en 2D

Antes de pasar a la red neuronal, se compararon los optimizadores en dos funciones clásicas: una **cuadrática mal condicionada** (número de condición $\kappa = 20$, rotada 30° para que los ejes no coincidan con las direcciones principales) y la **función de Rosenbrock** (valle curvo y estrecho, mínimo en (1, 1)). Para cada optimizador, la tasa de aprendizaje se eligió con una búsqueda en rejilla logarítmica de 25 valores entre 10^{-5} y 10, quedándose con la que daba el menor valor final de f. Momento y Nesterov usan $\mu = 0.9$ y Adam los valores por defecto.

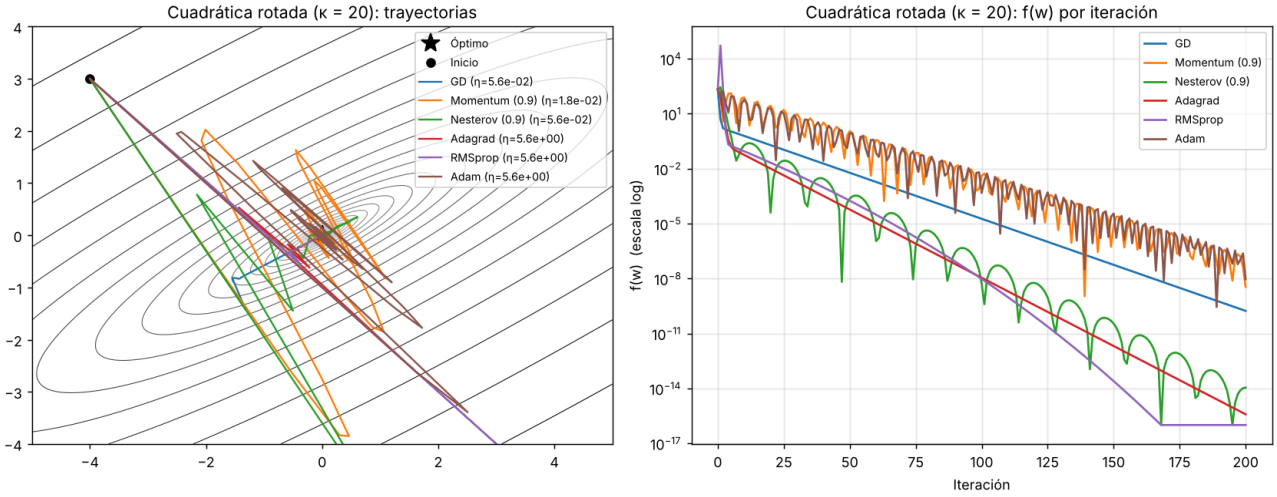


Figura 1. Cuadrática rotada (κ = 20), 200 iteraciones: trayectorias (izq.) y f(w) por iteración en escala logarítmica (der.).

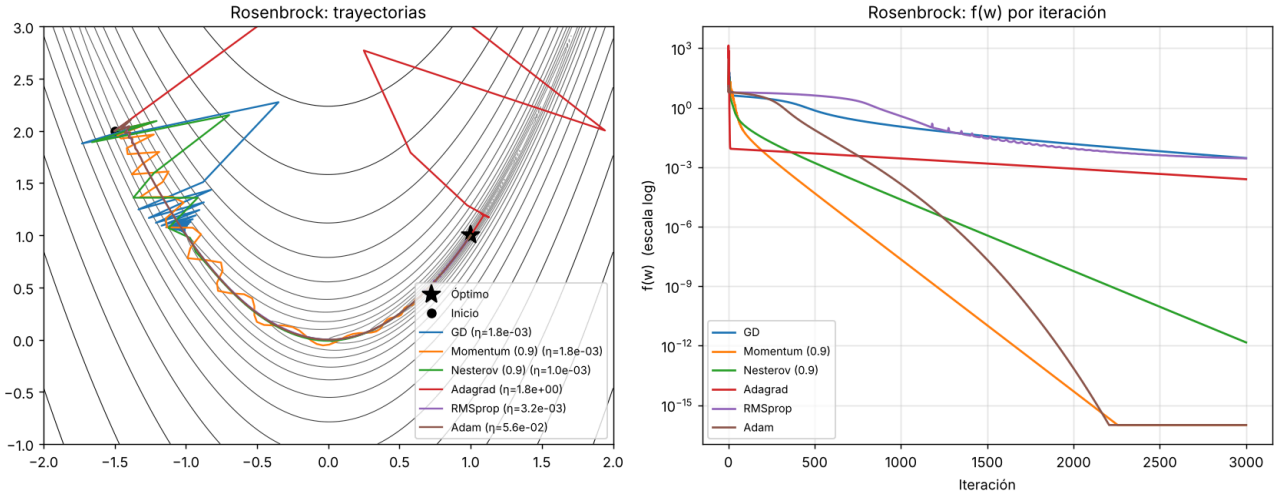


Figura 2. Función de Rosenbrock desde (−1.5, 2), 3 000 iteraciones.

Optimizador	**Cuadrática (200 it.)**		**Rosenbrock (3 000 it.)**	
	η elegida	**Iter. hasta $\|w - w^*\| < 10^{-3}$**	**η elegida**	**Iter. hasta $\|w - w^*\| < 10^{-3}$**
GD	5.6×10^{-2}	132	1.8×10^{-3}	no alcanza (dist. 0.12)
Momento (0.9)	1.8×10^{-2}	139	1.8×10^{-3}	**858**
Nesterov (0.9)	5.6×10^{-2}	**47**	1.0×10^{-3}	1 573
Adagrad	5.6	78	1.8	no alcanza (dist. 0.036)
RMSprop	5.6	82	3.2×10^{-3}	no alcanza (dist. 0.054)
Adam	5.6	136	5.6×10^{-2}	1 390

Observaciones preliminares.

- En la cuadrática, **Nesterov** es el más rápido. El momento clásico con μ = 0.9 resulta *más lento* que GD y oscila: para κ = 20 el valor teórico óptimo del momento (heavy ball) es $\mu^* = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2 \approx 0.40$,

así que $\mu = 0.9$ es demasiado alto. Esto muestra que el momento por sí solo no garantiza aceleración si no se ajusta al condicionamiento del problema.

- En Rosenbrock, los métodos con momento (momento clásico, Adam y Nesterov) son los únicos que llegan al mínimo, porque acumulan velocidad a lo largo del valle. GD, Adagrad y RMSprop avanzan muy despacio por el fondo del valle; Adagrad además se frena porque su acumulador G solo crece.
- Estas son funciones de juguete, deterministas y desde un único punto de partida, así que las conclusiones **no se trasladan directamente** a una red neuronal con gradientes estocásticos. Comprobarlo es justamente el objetivo de la semana II.

4.5 Archivos entregados

Archivo	Contenido
`semana1/optimizadores.py`	Implementación propia de SGD, momento, Nesterov, Adagrad, RMSprop, Adam y AdamW.
`semana1/test_optimizadores.py`	Validación numérica contra `torch.optim`.
`semana1/trayectorias_2d.py`	Experimento en funciones de prueba 2D.
`semana1/resultados/`	Figuras 1 y 2 y tablas CSV de resultados.
`train_resnet50.py`	Entrenamiento del modelo de referencia.
`app/`	Prototipo de la aplicación web (se completará en la semana V).